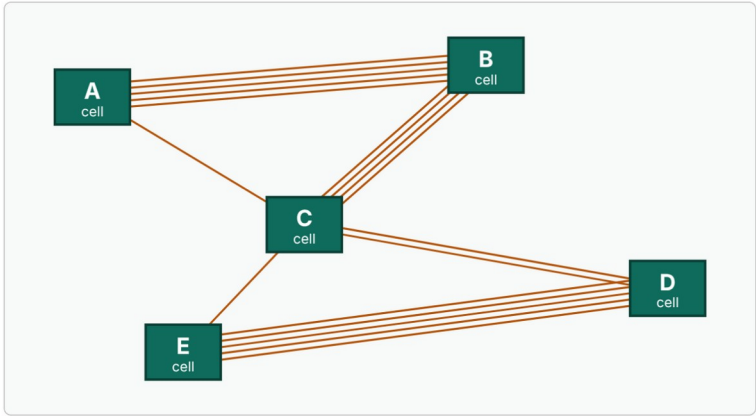
Affinity metrics

Which parts of a chip belong together?

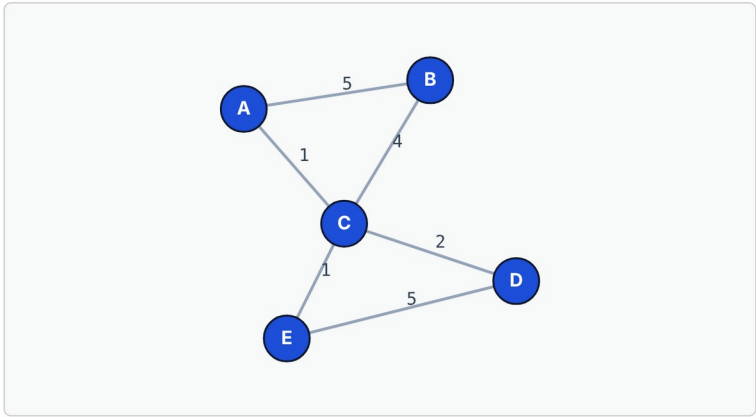
From a chip to a graph

- A chip contains cells, the small building blocks of the circuit
- Cells talk to each other through wires
- We draw each cell as a dot, called a node
- A line between two dots is an edge: it means the two cells are wired together
- The number on an edge is its weight: how many wires join the pair
- So A–B has weight five: five wires connect A and B

On the chip: cells joined by wires



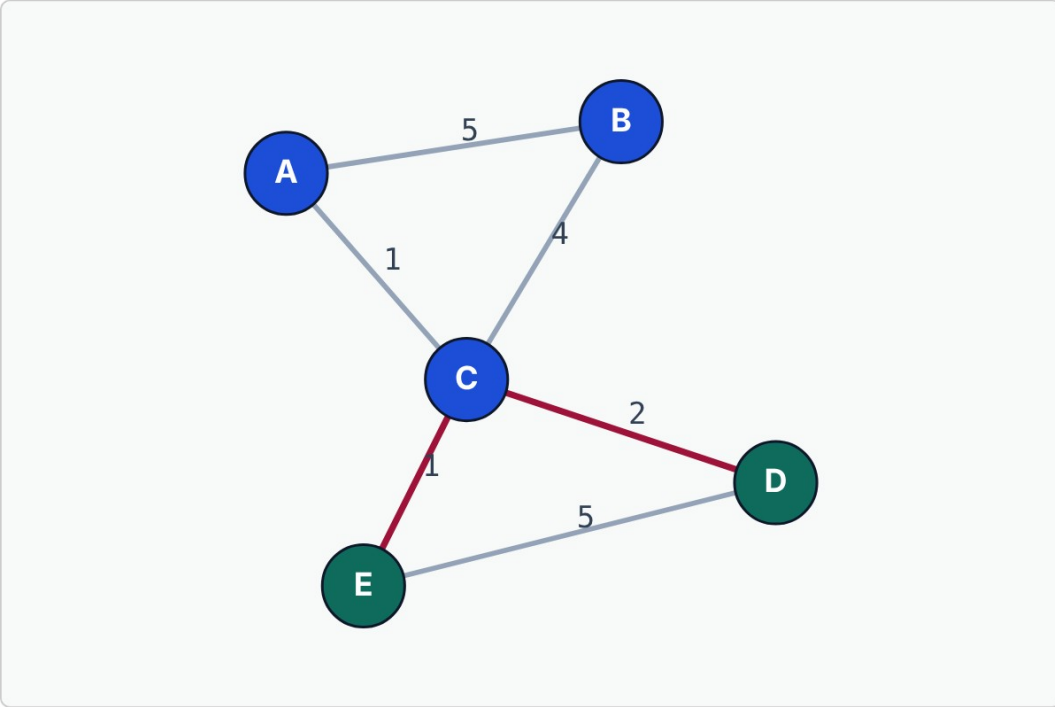
As a graph: weight = number of wires



Why group cells?

- Cells that exchange many signals should sit close together
- Short wires make the chip faster and save energy
- Grouping close cells is called clustering, and each group is a cluster
- Think of a seating plan: friends who chat a lot sit at the same table
- Here, A, B and C form one group, and D and E form another
- Only the red edges cross between the groups, and we want as few of those as possible

Two groups (clusters). Only the red edges cross between them.



What is affinity?

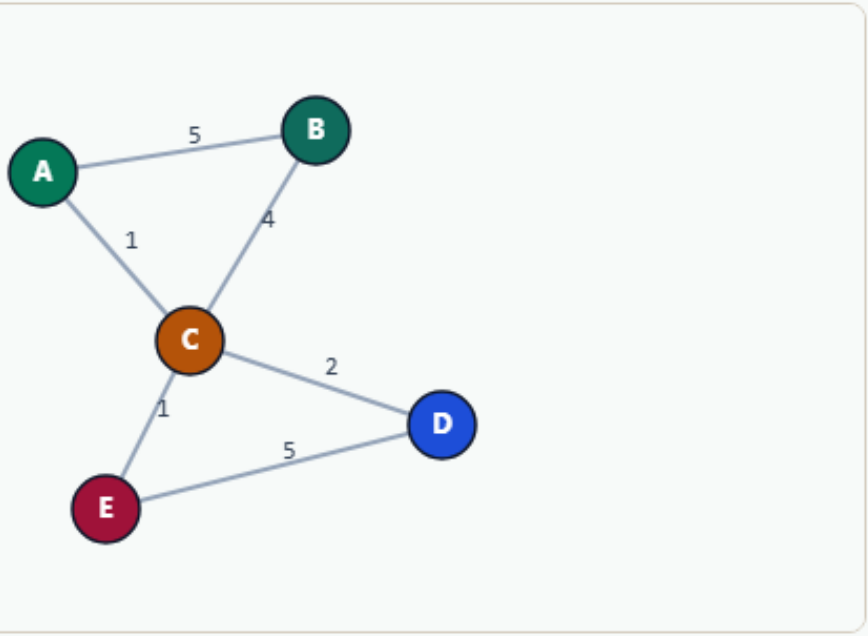
- Affinity is a score for each pair of cells
- A high score means these two should be in the same group
- A low score means they can be apart
- Clustering algorithms merge the pair with the highest score first
- So the recipe for the score decides everything that follows
- This lab compares three recipes on the same small example

Start with a tiny weighted graph

AFFINITY METRICS

Start with a tiny weighted graph

Step 1 / 5 · raw-graph · module01-01-affinity-metrics



Explanation

Five cells A–E with edge weights. Heavy edges A–B and D–E both weigh 5; the bridge through C is weaker. Affinity will decide which pairs want to cluster first.

- Nodes = cells / clusters to group
- Edge weight = how strongly a pair should stay together
- Reference instance used across all clustering labs

```
Graph: 5 nodes, 6 edges
Heaviest edges: A-B=5, D-E=5
```

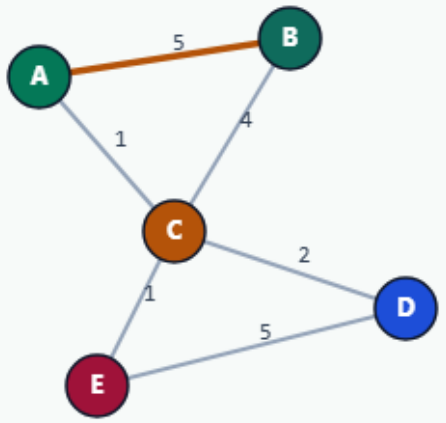
Start with a tiny weighted graph

Edge-weight affinity ranks pairs

AFFINITY METRICS

Edge-weight affinity ranks pairs

Step 2 / 5 · edge-rank · module01-01-affinity-metrics



Explanation

Pure edge-weight affinity just sorts existing edges. A–B and D–E tie at 5; tie-break puts A–B first. This ranking is the simplest merge priority.

- score(u,v) = w(u,v)
- Sort descending by score
- Top pair becomes the first merge candidate

```
1. A–B @ 5
2. D–E @ 5
3. B–C @ 4
4. C–D @ 2
5. A–C @ 1
6. C–E @ 1
```

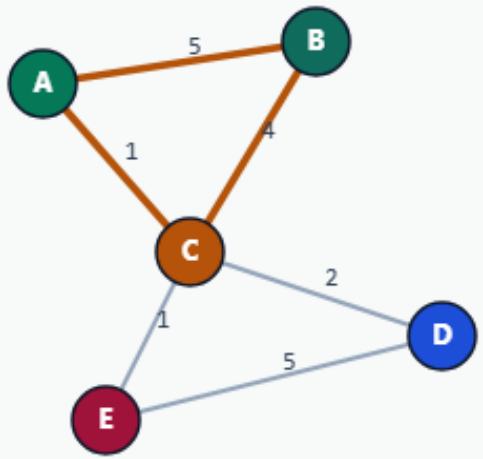
Edge-weight affinity ranks pairs

Shared neighbors boost A–B

AFFINITY METRICS

Shared neighbors boost A–B

Step 3 / 5 · shared-idea · module01-01-affinity-metrics



Explanation

A and B already share edge weight 5. They also share neighbor C. Shared-neighbor affinity adds min(w(A,C), w(B,C)) = min(1,4) = 1, so A–B rises to 6.

- Direct edge still counts
- Plus support through common neighbors
- Encourages dense triangles to cluster

```
w(A,B)=5
shared via C: min(1,4)=1
score(A,B)=6
```

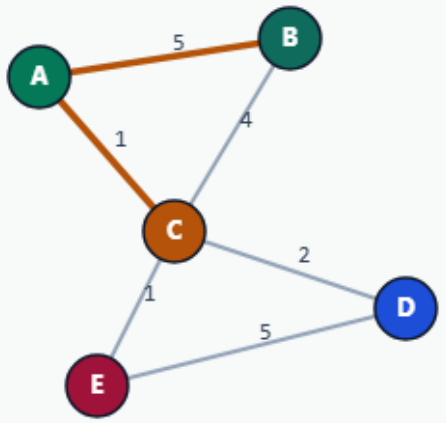
Shared neighbors boost A–B

Shared ranking reshuffles the list

AFFINITY METRICS

Shared ranking reshuffles the list

Step 4 / 5 · shared-rank · module01-01-affinity-metrics



Explanation

Under shared-neighbor scoring, A–B and D–E lead at 6. Weak edge A–C jumps to 5 because of B. Non-edges like B–D can appear from shared support alone.

- A–C: was 1 → now 5
- Invented pairs (e.g. B–D @ 2) show transitive pull
- Same idea feeds greedy merge priorities

```
1. A–B @ 6
2. D–E @ 6
3. A–C @ 5
4. B–C @ 5
5. C–D @ 3
6. C–E @ 3
```

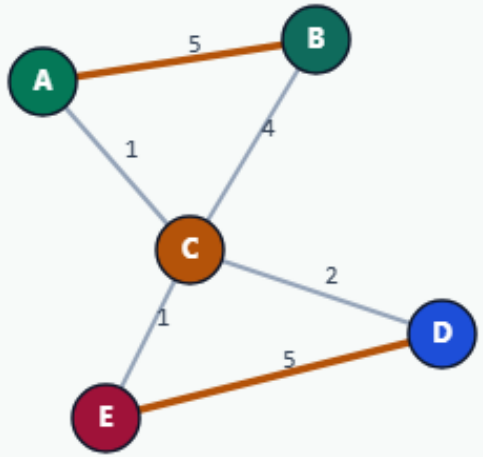
Shared ranking reshuffles the list

Why affinity matters in EDA

AFFINITY METRICS

Why affinity matters in EDA

Step 5 / 5 · takeaway · module01-01-affinity-metrics



Explanation

Affinity is the objective's local preference: which cells belong together before global constraints. Bad affinity → bad merges → harder refinement later.

- Edge weight = connectivity / nets shared
- Shared neighbors ≈ denser local groups
- Next labs consume this ranking to merge

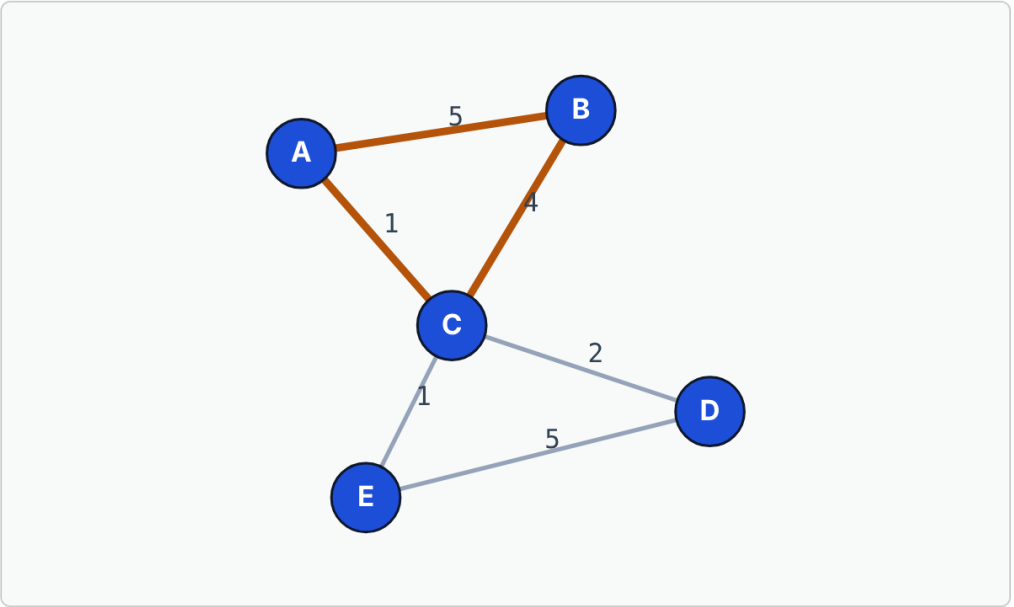
```
Edge top: A–B @ 5
Shared top: A–B @ 6
Teaching graph stays fixed for goldens
```

Why affinity matters in EDA

Worked example: the score of A–C

- Let's compute one score together, step by step
- Step one: the direct edge A–C has weight one
- Step two: A and C have one neighbor in common, B
- The links through B weigh five and four, so we keep the smaller one: four
- Step three: add them up, one plus four makes five
- A–C was weak alone, but becomes strong once its shared friend counts

Score of A–C, step by step



1. Direct edge A–C: 1
2. Common neighbor B: min(5, 4) = 4
3. **Total: 1 + 4 = 5**

Ten pairs, not six

- With recipe two, our small graph gives ten pairs instead of six
- Four of them have no wire at all between them
- For example, B and D are not wired together, but both are wired to C
- So B–D gets the weaker of those two links: two
- It's like two people who have never met but share a best friend
- Only pairs whose score would be zero are left out

Recipe two as pseudocode

- Here is recipe two written as steps a computer can follow
- First, every edge adds its weight to its own pair
- Next, for every pair of nodes, look at each neighbor they have in common
- For each shared neighbor, add the smaller of its two links to the pair
- Keep every pair whose score ends up above zero
- Finally, sort from highest to lowest score, and break ties alphabetically

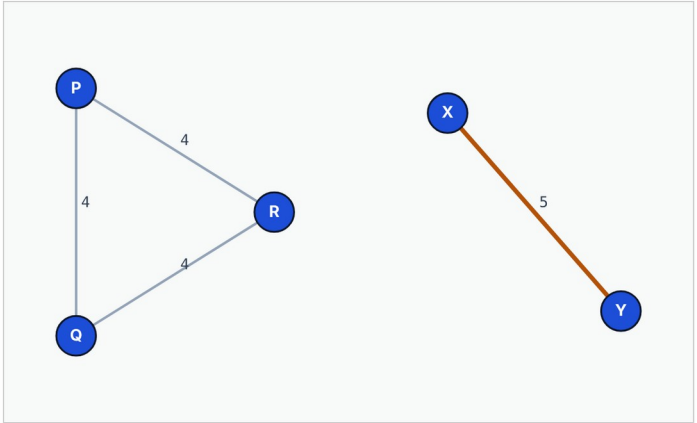
Recipe two as pseudocode — try these

```
affinity_shared_neighbors(nodes, edges):
  score[u,v] = w(u,v) for every edge
  for every pair (u, v) of nodes:
    s = 0
    for x in neighbors(u), x != v:
      if x in neighbors(v):
        s += min(w(u,x), w(v,x))
    if s > 0 or (u,v) has an edge:
      score[u,v] += s
  sort by score (high first), then u, then v
```

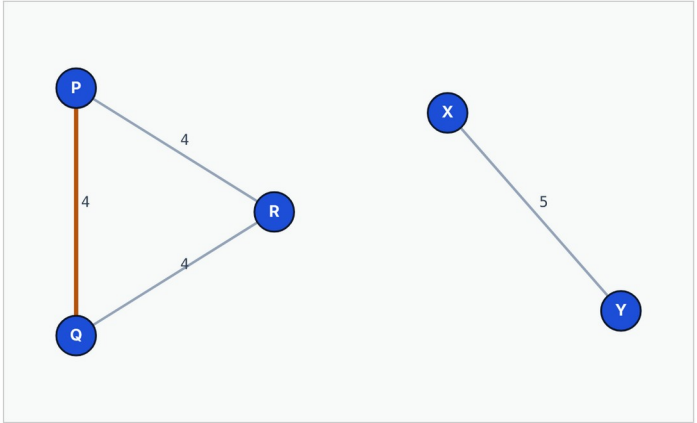
Same graph, different first merge

- Here's a second example that shows why the recipe matters
- Take a tight triangle P, Q, R, with every edge at weight four
- Next to it, a lone pair X–Y has weight five
- Recipe one picks X–Y first, because five beats four
- Recipe two gives P–Q four, plus four through R: eight, so P–Q goes first
- Same graph, same merge rule, but a different first merge

Edge weight: first merge X–Y (5)



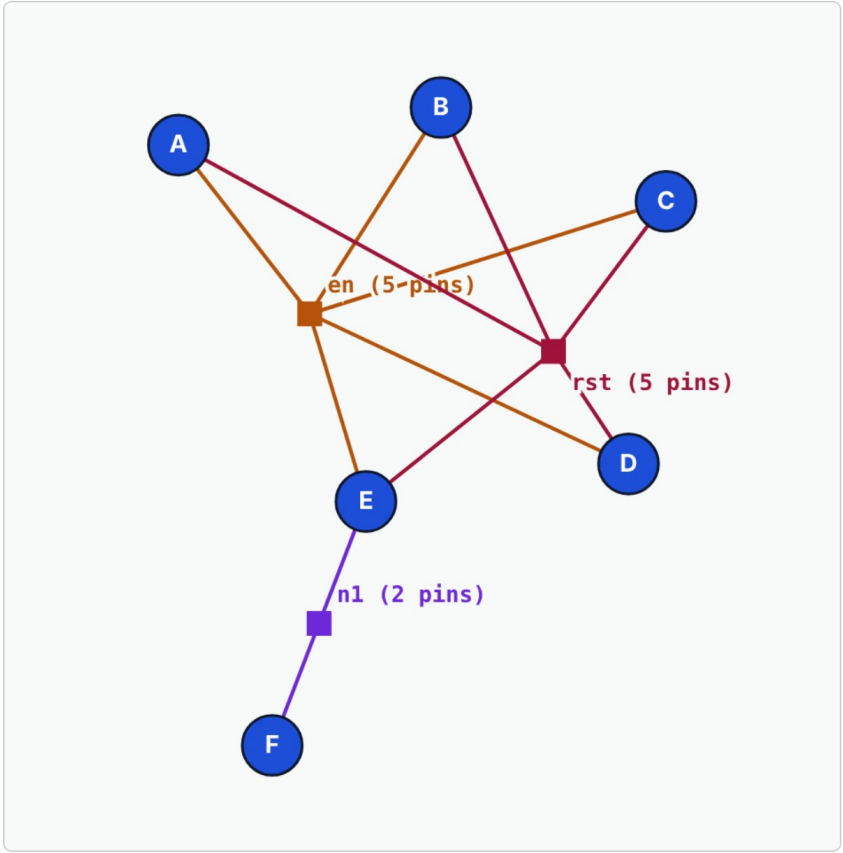
Shared neighbors: first merge P–Q (4+4 = 8)



Nets with many pins

- On a real chip, one wire can connect many cells at once
- Such a shared wire is called a net, and each cell it touches is a pin
- Think of a group chat with fifty people: it says little about who is friends
- A private message between two people says much more
- Recipe three works the same way: each net shares its weight among its pins
- The rule is the weight divided by the number of pins, minus one

Two wide nets (en, rst) touch five cells. One private net joins E and F.

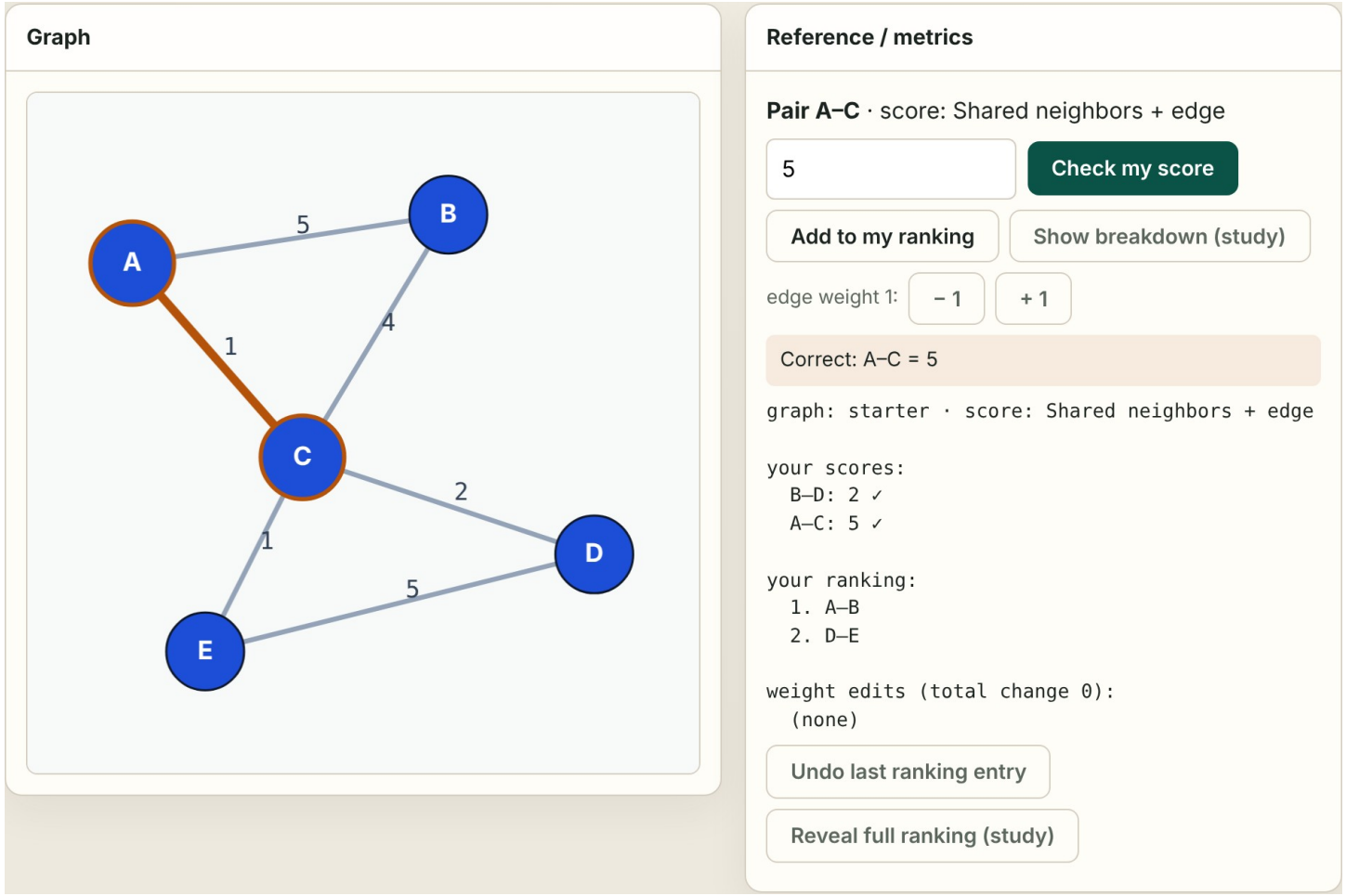


Nets with many pins — try these

```
tiny netlist: en {A,B,C,D,E}, rst {A,B,C,D,E}, n1 {E,F}

           raw net count     pin-count w/(|e|-1)
A-B        1 + 1 = 2         1/4 + 1/4 = 0.5
E-F        1                 1/1 = 1.0
first      A-B               E-F
```

Browser lab track



Browser lab starter

Implement track

wsl — module01-01-affinity-metrics/examples/

```
# Track A — real Unix

$ python3 ../common/solvers.py examples/priority_graph.json --mode affinity
affinity edge-weight (desc):
  X-Y: 5.0
  P-Q: 4.0
  P-R: 4.0
  Q-R: 4.0
affinity shared-neighbors+edge (desc):
  P-Q: 8.0
  P-R: 8.0
  Q-R: 8.0
  X-Y: 5.0

$ python3 examples/affinity_check.py
checking reference
PASS  tiny graph, edge weight: 6 pairs in golden order
PASS  tiny graph, shared neighbors: 10 pairs in golden order
PASS  priority graph, edge weight: first merge X-Y @ 5
PASS  priority graph, shared neighbors: first merge P-Q @ 8
PASS  netlist, pin-count: first merge E-F @ 1, A-B only 0.5
all goldens match
```

Real shell session

Implement track — try these

```
# point Python at the shared helpers
export PYTHONPATH=../common

# both graph rankings, starter graph then priority graph
python3 ../common/solvers.py examples/tiny_graph.json --mode
affinity
python3 ../common/solvers.py examples/priority_graph.json --mode
affinity

# check every golden against your own functions
python3 examples/affinity_check.py --impl my_affinity.py
```

Pitfalls to watch

- Watch out for four common mistakes
- Counting an edge twice, once from each end
- Adding the stronger link instead of the weaker one, which makes every pair look close
- Forgetting the tie-break, so A–B and D–E swap places from one run to the next
- And counting big shared nets at full weight, which glues unrelated cells together

Recap: three recipes

- Recipe one, edge weight: look only at the direct wire
- Recipe two, shared neighbors: add the help of common friends
- Recipe three, pin count: big shared nets count less than private ones
- Ties are broken alphabetically, so results never change between runs
- Different recipes can choose a different first merge
- Choosing the recipe is a design decision, not a detail

Your turn

- Try the browser lab challenges, or write the three recipes yourself and pass the checker
- Then explain two things in your own words
- First, why does A–C climb with recipe two?
- Second, why does the priority graph change its first merge?
- Take the short quiz, then move on to greedy pair merge
- There, these scores become the rule for merging